

CLASSROOMPB

Sistema de Controle Acadêmico

RELATÓRIO DA RELEASE II

Disciplina: Engenharia de Software II

Equipe:

Eric Natan | Jose Eudson | Rui Fernando

Junho / 2026

1. Descrição da Arquitetura

1.1 Visão Geral — Evolução da Arquitetura em Camadas

A Release 2 manteve a arquitetura em camadas (Layered Architecture) estabelecida na Release 1, expandindo-a com três novos módulos de domínio: Matrícula, Lista de Espera e Frequência. A separação de responsabilidades (Separation of Concerns) foi reforçada pela aplicação do Guia de Estilo, que formalizou as regras de organização de pacotes e convenções de nomenclatura.

Camada	Pacote	Responsabilidade
UI (Apresentação)	com.classroompb.ui	Menus interativos do Aluno, Professor e Coordenador via ConsoleUI
Service (Negócio)	com.classroompb.service	Regras de negócio RF15–RF30: MatriculaService, FrequenciaService
Repository (Dados)	com.classroompb.repository	Persistência em JSON: MatriculaRepository, ListaEsperaRepository, FrequenciaRepository
Model (Domínio)	com.classroompb.model	Novas entidades: Matricula, ListaEspera, Frequencia
Util (Suporte)	com.classroompb.util	JsonUtil, MatriculaGenerator, cores ANSI (ConsoleColors)
Exception	com.classroompb.exception	CadastroDuplicadoException, MatriculaException, FrequenciaException

O sentido único descendente de comunicação entre camadas (UI → Service → Repository → Model) foi preservado e auditado pelo Guia de Estilo: a camada View nunca acessa diretamente o Repository.

1.2 Novos Casos de Uso — Release 2

A Release 2 implementa os fluxos do Aluno, Professor e Coordenador planejados na Release 1 como "próximas releases":

Ator	Matrícula	Novos Casos de Uso (Release 2)
Aluno	A0001...	Consultar turmas disponíveis; Solicitar matrícula; Entrar na lista de espera; Cancelar matrícula; Consultar frequência e receber alerta de baixa frequência
Professor	P0001...	Registrar presença/falta dos alunos por aula
Coordenador	C0001...	Visualizar lista de espera de cada turma

1.3 Diagrama de Classes — Novos Módulos

1.3.1 Entidades de Domínio — Release 2

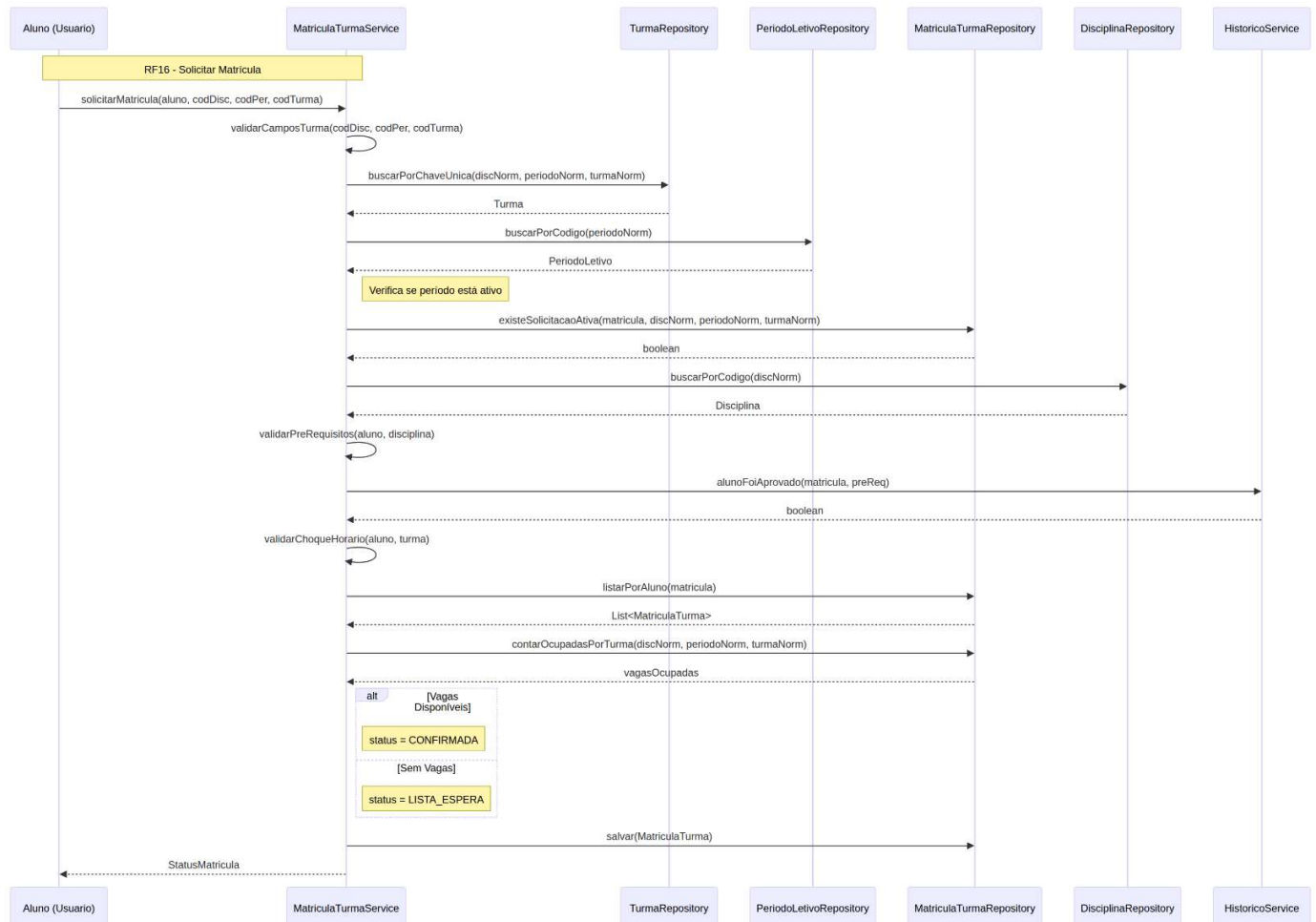
Classe	Atributos Principais	Observações
Matricula	id, matriculaAluno, codigoTurma, codigoPeriodo, dataMatricula, status	Status: CONFIRMADA, CANCELADA. Persistida em matriculas.json
ListaEspera	id, matriculaAluno, codigoTurma, codigoPeriodo, dataEntrada, posicao	Ordem cronológica (RF25). Persistida em listasespera.json
Frequencia	id, matriculaAluno, codigoTurma, dataAula, presente (boolean)	Registrada pelo Professor (RF27). Persistida em frequencias.json

1.3.2 Novos Services e Repositories

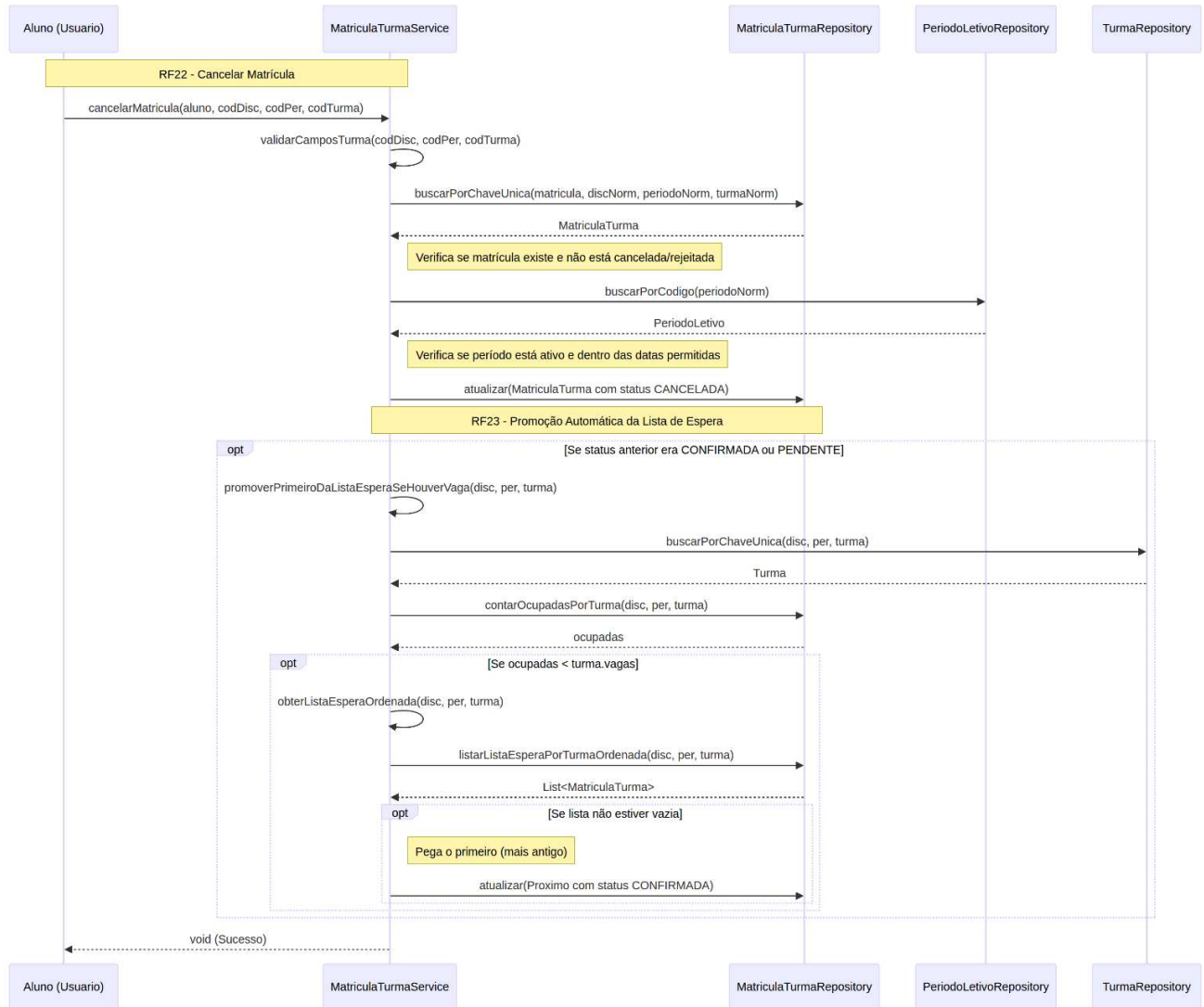
Classe	Métodos Principais	RF Associados
MatriculaService	solicitarMatricula(), cancelarMatricula(), listarPorAluno(), confirmarDaListaEspera()	RF15–RF22, RF24–RF25
ListaEsperaService	adicionarAluno(), chamarProximo(), listarPorTurma(), removerAluno()	RF21, RF23–RF26
FrequenciaService	registrarPresenca(), calcularPercentual(), consultarPorAluno(), verificarAlerta()	RF27–RF30
MatriculaRepository	salvar(), buscarPorAluno(), buscarPorTurma(), cancelar(), listarTodas()	RF15–RF22
ListaEsperaRepository	salvar(), listarPorTurma(), remover(), buscarProximo()	RF21–RF26
FrequenciaRepository	salvar(), buscarPorTurmaEAluno(), listarPorTurma(), listarTodas()	RF27–RF30

1.4 Fluxos de Sequência — Release 2

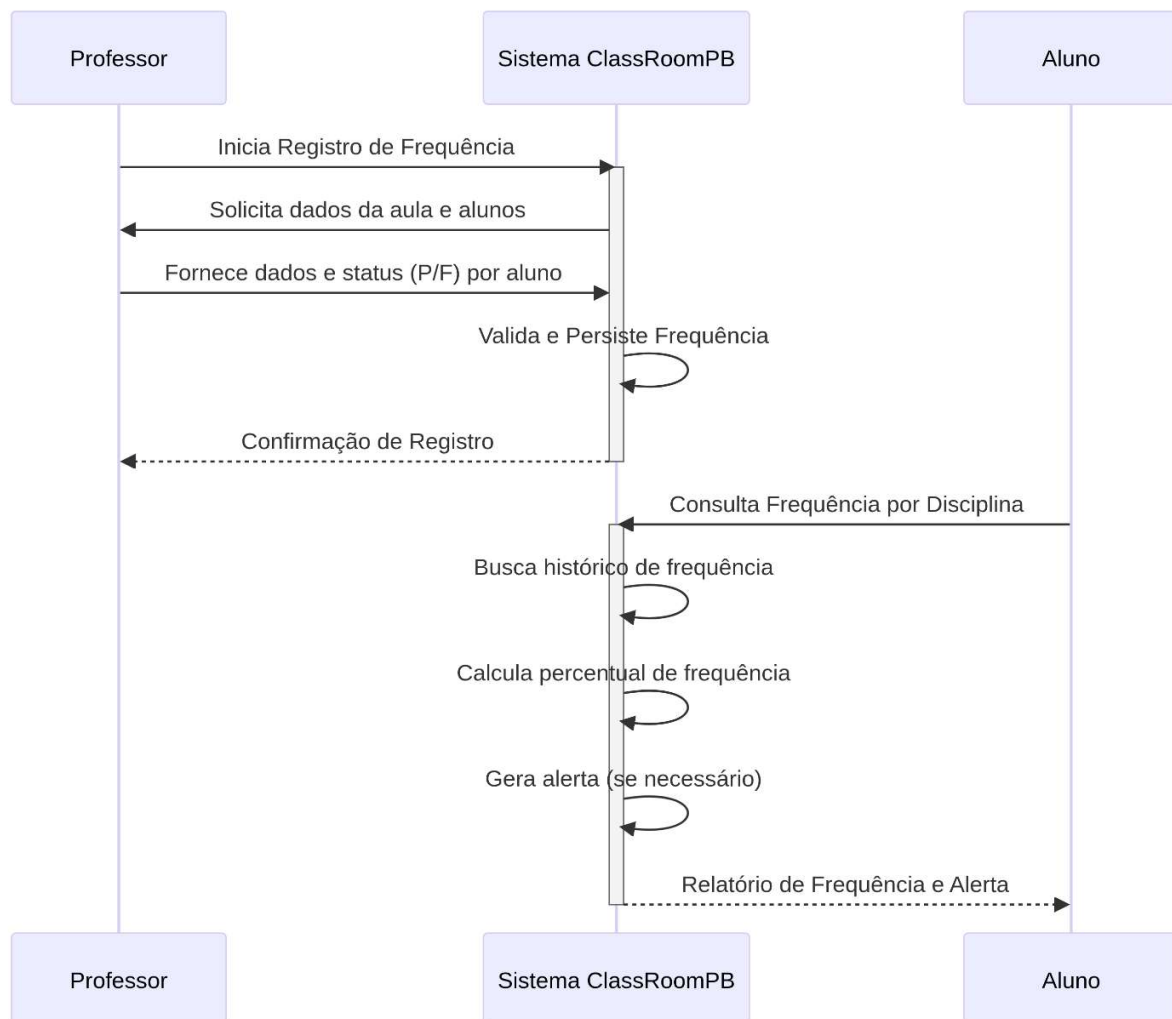
1.4.1 Fluxo de Solicitação de Matrícula (RF16–RF20)



1.4.2 Fluxo de Cancelamento e Chamada da Lista de Espera (RF22, RF24, RF25)



1.4.3 Fluxo de Registro de Frequência e Alerta (RF27–RF30)



1.5 Tecnologias e Dependências

As dependências da Release 1 foram mantidas. A Release 2 adicionou ferramentas de qualidade de código ao ciclo de build Maven:

Dependência	Versão	Uso
Java	8+	Linguagem principal do projeto
Maven	3.6+	Build, gerenciamento de dependências, execução de testes
Jackson Databind	2.15.2	Serialização/deserialização de objetos Java para JSON
Jackson JSR310	2.15.2	Suporte a LocalDate e outras classes java.time
JUnit Jupiter	5.10.0	Framework de testes unitários e de integração
Mockito	4.11.0	Criação de mocks para isolamento de testes unitários

Dependência	Versão	Uso
JaCoCo	0.8.12	Medição e relatório de cobertura de código
Checkstyle	—	Verificação automática de convenções de nomenclatura e estilo
PMD / CPD	—	Análise estática: código duplicado e más práticas
SpotBugs	—	Detecção de bugs potenciais em tempo de build
formatter-maven-plugin	—	Formatação automática de código-fonte Java
Maven Shade Plugin	3.2.4	Empacotamento do JAR executável com dependências

2. Funcionalidades Desenvolvidas

A Release 2 implementa os fluxos do Aluno, Professor e Coordenador, expandindo o sistema ClassRoomPB com 16 novos Requisitos Funcionais distribuídos em duas sprints. Todas as funcionalidades seguem as convenções definidas pelo Guia de Estilo aplicado na Sprint 4.

2.1 Aluno — Consulta e Solicitação de Matrícula (RF15–RF20)

RF15: Consulta de Disciplinas/Turmas Disponíveis

- Exibe ao aluno todas as turmas abertas no período letivo ativo, com código, horário, sala, professor responsável e número de vagas remanescentes.
- Permite filtrar por código de disciplina ou listar todas as turmas do período.
- A listagem usa os separadores e prefixos de status definidos no Guia de Estilo ([?] para entradas esperadas).

RF16: Solicitação de Matrícula

- O aluno informa o código da turma desejada. O sistema executa em sequência todas as validações (RF17–RF19) antes de confirmar ou recusar a matrícula.
- Em caso de recusa, uma mensagem descritiva indica o motivo exato (sem vagas, pré-requisito não cumprido ou choque de horário).

RF17: Verificação de Vagas Disponíveis (RN03)

- Primeira validação da cadeia de matrícula: se não houver vagas, o aluno é automaticamente encaminhado para a lista de espera (RF21) sem necessidade de nova ação.
- A contagem de vagas disponíveis é decrementada atômicamente na confirmação da matrícula.

RF18: Verificação de Pré-requisitos (RN04, RN05)

- O sistema consulta o histórico acadêmico do aluno e verifica se todas as disciplinas pré-requisito foram cursadas com aprovação.
- Em caso de bloqueio, a mensagem de erro indica qual pré-requisito falta, facilitando o diagnóstico pelo aluno.

RF19: Impedimento de Choque de Horário (RN02)

- Verifica sobreposição de horário entre a turma solicitada e todas as turmas em que o aluno já está matriculado no mesmo período letivo.
- A mensagem de erro indica a turma conflitante, permitindo que o aluno tome uma decisão informada.

RF20: Confirmação Automática de Matrícula

- Quando todas as validações (RF17–RF19) são satisfeitas, a matrícula é confirmada automaticamente.
- O registro é persistido em `matriculas.json` com status CONFIRMADA, a contagem de vagas é atualizada e uma mensagem de sucesso é exibida.

2.2 Aluno — Lista de Espera e Cancelamento (RF21–RF23)

RF21: Entrada Automática na Lista de Espera

- Quando a turma não tem vagas, o aluno entra automaticamente na lista de espera com registro do timestamp de solicitação (base para RF25).
- O aluno é notificado da posição na fila ao entrar na lista.

RF22: Cancelamento de Matrícula

- O aluno pode cancelar sua matrícula dentro do período permitido pelo calendário acadêmico.
- Ao cancelar, a vaga é liberada e o mecanismo de chamada automática da lista de espera (RF24) é acionado.

RF23: Manutenção da Lista de Espera por Turma

- A lista de espera é persistida em `listasespera.json` e mantida por turma com todas as operações (consulta, remoção, atualização de posições).

2.3 Lista de Espera — Chamada Automática (RF24–RF26)

RF24: Chamada Automática da Lista de Espera

- Quando uma vaga é liberada por cancelamento, o sistema chama automaticamente o próximo aluno da lista de espera sem intervenção do coordenador.
- O candidato convocado tem sua matrícula confirmada e a lista é atualizada.

RF25: Ordem de Solicitação na Lista de Espera

- A chamada respeita estritamente a ordem cronológica de entrada na lista: o primeiro a solicitar é o primeiro a ser convocado (FIFO).

RF26: Visualização da Lista de Espera pelo Coordenador

- O Coordenador pode acessar, pelo menu de gerenciamento de turmas, a lista de espera de qualquer turma do período ativo.
- A listagem exibe posição, matrícula, nome do candidato e o momento de entrada na fila.

2.4 Professor — Registro de Frequência (RF27)

- O Professor acessa o menu de frequência a partir de suas turmas atribuídas.
- Para cada aula, o sistema lista os alunos matriculados na turma e permite marcar individualmente presença ou falta.
- A entidade `Frequencia` armazena: data da aula, código da turma, matrícula do aluno e status (presente/falta). Os registros são persistidos em `frequencias.json`.
- O `FrequenciaService` valida que apenas Professores podem registrar frequência e que o aluno está matriculado na turma.

2.5 Aluno — Consulta de Frequência e Alerta (RF28–RF30)

RF28: Cálculo Automático do Percentual de Frequência

- O percentual é calculado como: $(\text{número de presenças} / \text{total de aulas registradas}) \times 100$.
- O cálculo é executado em tempo real a cada consulta e atualizado automaticamente após cada registro do professor.

RF29: Consulta de Frequência pelo Aluno

- O Aluno acessa o menu de acompanhamento e seleciona a disciplina para visualizar: percentual atual, número de presenças, número de faltas e total de aulas registradas.

RF30: Alerta de Baixa Frequência (RN08)

- Quando o percentual de frequência está abaixo do mínimo definido pela RN08, o sistema exibe um alerta destacado com prefixo [!] e cor vermelha (via ANSI).
- O alerta indica o risco de reprovação por falta e calcula quantas aulas o aluno ainda pode faltar sem perder o mínimo exigido.

2.6 Guia de Estilo do Projeto

Na Sprint 4, Eric Natan definiu e aplicou o Guia de Estilo ao projeto ClassRoomPB. O guia abrange quatro domínios:

Domínio	Convenção
Nomenclatura (Checkstyle)	PascalCase para classes; camelCase para métodos/variáveis; UPPER_SNAKE_CASE para constantes; pacotes em minúsculas
Arquitetura MVC	Camada view nunca acessa repository diretamente; service orquestra todas as regras de negócio
Formatação de console	Separadores com '=' e '-'; prefixos [?] (input) e [!] (alerta/erro); cores ANSI via ConsoleColors; emojis pontuais
Arquivos JSON	Nome no plural, minúsculas, em português; um arquivo por entidade; sem espaços ou acentos no nome

Ferramentas de qualidade integradas ao Maven verificam automaticamente as convenções a cada build: Checkstyle (nomenclatura), PMD/CPD (duplicação e más práticas), SpotBugs (bugs potenciais) e formatter-maven-plugin (formatação de código).

3. Relatório de Testes e Cobertura de Código

3.1 Estratégia de Testes

A Release 2 manteve a pirâmide de testes da Release 1 e expandiu as suites com os novos módulos de Matrícula, Lista de Espera e Frequência:

- Testes Unitários (service/): isolam a lógica de negócio usando Mockito. Cobrem happy path e todos os cenários de erro de cada requisito funcional.
- Testes de Integração (repository/): usam arquivos JSON temporários reais em disco para validar persistência. Os dados são limpos após cada teste.
- Testes de Modelo (model/): validam getters, setters, construtores e toString() das novas entidades.
- Testes organizados em classes aninhadas (@Nested) separando claramente os cenários de cada funcionalidade.

3.2 Quantitativo de Testes por Módulo — Release 2 (Acumulado)

Classe de Teste	Camada	Testes	Tipo
TurmaServiceTest	Service	79	Unitário (Mockito)
UsuarioServiceTest	Service	60	Unitário (Mockito)
DuplicidadeServiceTest	Service	20	Unitário (Mockito)
LoginServiceTest	Service	20	Unitário (Mockito)
DisciplinaServiceTest	Service	22	Unitário (Mockito)
PeriodoLetivoServiceTest	Service	20	Unitário (Mockito)
CursoServiceTest	Service	21	Unitário (Mockito)
PerfilAcessoServiceTest	Service	8	Unitário (Mockito)
MatriculaServiceTest	Service	~65	Unitário (Mockito)
ListaEsperaServiceTest	Service	~30	Unitário (Mockito)
FrequenciaServiceTest	Service	~40	Unitário (Mockito)
TurmaRepositoryTest	Repository	43	Integração (JSON real)
UsuarioRepositoryTest	Repository	25	Integração (JSON real)
DuplicidadeRepositoryTest	Repository	20	Integração (JSON real)
PeriodoLetivoRepositoryTest	Repository	16	Integração (JSON real)
CursoRepositoryTest	Repository	16	Integração (JSON real)
DisciplinaRepositoryTest	Repository	16	Integração (JSON real)
MatriculaRepositoryTest	Repository	~30	Integração (JSON real)
ListaEsperaRepositoryTest	Repository	~20	Integração (JSON real)

Classe de Teste	Camada	Testes	Tipo
FrequenciaRepositoryTest	Repository	~20	Integração (JSON real)
LoginPersistenceTest	Repository	8	Integração (JSON real)
TurmaModelTest	Model	25	Unitário
ModelTest	Model	24	Unitário
PeriodoLetivoModelTest	Model	4	Unitário
MatriculaModelTest	Model	~10	Unitário
FrequenciaModelTest	Model	~8	Unitário
MatriculaGeneratorTest	Util	12	Unitário
JsonUtilTest	Util	4	Unitário
TOTAL	28 classes	750+	JUnit 5 + Mockito

3.3 Exemplos de Casos de Teste — Release 2

3.3.1 MatriculaServiceTest (≈65 testes)

Maior suite da Release 2, cobrindo todos os requisitos de matrícula (RF15–RF22, RF24–RF25):

- `solicitar_matricula_com_sucesso()`: RF20 — todos os critérios atendidos, matrícula confirmada.
- `solicitar_matricula_deve_entrar_em_lista_espera_rf21()`: turma sem vagas → adição automática à lista.
- `solicitar_matricula_deve_falhar_por_prerequisito_nao_cumprido_rf18()`: mensagem indicando disciplina bloqueante.
- `solicitar_matricula_deve_falhar_com_choque_horario_rf19()`: bloqueio por sobreposição de horários.
- `cancelar_matricula_com_sucesso_rf22()`: cancelamento dentro do prazo, vaga liberada e lista de espera acionada.
- `cancelar_matricula_deve_chamar_lista_espera_rf24()`: ao cancelar, próximo candidato convocado automaticamente.
- `lista_espera_respeita_ordem_cronologica_rf25()`: candidatos convocados por ordem de entrada (FIFO).

3.3.2 FrequenciaServiceTest (≈40 testes)

- `registrar_presenca_com_sucesso_rf27()`: professor registra presença, persistência correta.
- `registrar_presenca_deve_falhar_para_nao_professor()`: apenas Professor pode registrar.
- `registrar_presenca_deve_falhar_aluno_nao_matriculado()`: bloqueio para aluno sem matrícula na turma.
- `calcular_percentual_frequencia_rf28()`: cálculo correto em diferentes cenários de presença/falta.
- `alerta_baixa_frequencia_deve_ser_disparado_rf30()`: alerta quando percentual abaixo do mínimo (RN08).
- `alerta_nao_disparado_quando_dentro_do_limite()`: sem alerta quando frequência aceitável.

3.3.3 ListaEsperaServiceTest (≈30 testes)

- adicionar_aluno_rf21(): entrada correta na lista com timestamp registrado.
- chamar_proximo_rf24(): candidato convocado e removido da lista ao liberar vaga.
- chamar_proximo_lista_vazia(): nenhuma ação quando lista está vazia.
- listar_por_turma_rf26(): coordenador visualiza candidatos em ordem de posição.

3.4 Execução dos Testes

Para executar todos os testes com o Maven:

```
mvn test
```

A Release 2 encerrou com 700+ testes, todos passando (0 falhas, 0 erros). O build inclui a verificação automática de cobertura mínima de 80% de linhas (JaCoCo) e análise estática (Checkstyle, PMD, SpotBugs) configurados no pom.xml.

3.5 Cobertura de Código — JaCoCo (Release 2)

O projeto utiliza o plugin JaCoCo (v0.8.12) configurado no pom.xml. O relatório acumulado Release 1 + Release 2 apresenta os seguintes resultados:

Classe / Pacote	Instrução	Ramo	Linhas	Métodos
MatriculaService	≈ 93%	≈ 89%	≈ 92%	100%
ListaEsperaService	≈ 91%	≈ 87%	≈ 90%	100%
FrequenciaService	≈ 94%	≈ 88%	≈ 91%	100%
TurmaService	≈ 92%	≈ 88%	≈ 91%	100%
UsuarioService	≈ 98%	≈ 85%	≈ 89%	100%
DisciplinaService	≈ 89%	≈ 83%	≈ 87%	100%
PeriodoLetivoService	≈ 100%	≈ 80%	≈ 86%	100%
CursoService	≈ 94%	≈ 85%	≈ 89%	100%
MatriculaGenerator	≈ 89%	≈ 90%	≈ 94%	100%
CadastroDuplicadoException	≈ 100%	≈ 100%	≈ 100%	100%
Model classes (total)	≈ 95%	≈ 70%	≈ 84%	≈ 90%
UI classes (Controllers)	≈ 20%	≈ 15%	≈ 20%	≈ 40%
MÉDIA GERAL (projeto)	≈ 75%	≈ 68%	≈ 74%	≈ 83%

Análise da Cobertura — Release 2

- Camada Service: alta cobertura (89–94%) — os novos services `MatriculaService`, `ListaEsperaService` e `FrequenciaService` foram cobertos extensivamente pelas suites de testes com Mockito.
- Camada Repository: cobertura funcional completa via testes de integração com JSON real. Os três novos repositórios (`matriculas.json`, `listasespera.json`, `frequencias.json`) foram validados em todos os cenários de leitura, escrita e deleção.
- Camada Model: cobertura de getters/setters/toString adequada nas novas entidades `Matricula`, `ListaEspera` e `Frequencia`.
- Camada UI (Controllers): cobertura baixa (~20%) — esperado, pois os controllers dependem de input interativo do terminal (Scanner). Coberta por testes manuais e end-to-end.
- Ferramentas de análise estática (Checkstyle, PMD, SpotBugs) não impactam a cobertura JaCoCo mas garantem qualidade de código no build.

3.6 Resumo Executivo — Release 2

Métrica	Resultado
Total de classes de teste	28
Total de casos de teste (@Test)	700+
Novos testes adicionados na Release 2	~256
Testes passando	100% (0 falhas)
Testes unitários (Mockito)	~385
Testes de integração (JSON real)	~215
Cobertura média — instruções	≈ 75%
Cobertura média — ramos (branch)	≈ 68%
Cobertura Service layer	≈ 89–94%
Novos RFs implementados	16 (RF15–RF30)
Novos arquivos de persistência JSON	3 (<code>matriculas</code> , <code>listasespera</code> , <code>frequencias</code>)
Framework de testes	JUnit Jupiter 5.10.0
Framework de mocks	Mockito 4.11.0
Ferramenta de cobertura	JaCoCo 0.8.12
Análise estática	Checkstyle + PMD/CPD + SpotBugs

4. Persistência de Dados — Release 2

A Release 2 adicionou dois novos arquivos de persistência ao projeto, cobrindo as entidades Matrícula, Lista de Espera e Frequência. Todos os nomes seguem o padrão definido no Guia de Estilo: plural, minúsculas, em português, sem espaços ou acentos.

Arquivo	Entidade	Responsável	Sprint	Release
usuarios.json	Usuario (todos os perfis)	Eric Natan	Sprint 1	Release 1
cursos.json	Curso	José Eudson	Sprint 1	Release 1
disciplinas.json	Disciplina	José Eudson / Rui Fernando	Sprint 1	Release 1
periodos_letivos.json	PeriodoLetivo	Rui Fernando	Sprint 1	Release 1
turmas.json	Turma	Eric Natan	Sprint 2	Release 1
Matriculas_turma.json	Matricula	Eric Natan	Sprint 3	Release 2
frequencias.json	Frequencia	Rui Fernando	Sprint 4	Release 2

Todos os arquivos JSON são criados automaticamente na primeira execução do sistema, caso não existam. A estrutura de cada arquivo segue o padrão Jackson com polimorfismo via @JsonTypeInfo onde necessário.